

Combining Instance Filtering and Attribute Weighting in Decision Tree and Naïve Bayes Hybrids

Ikramul Hasan Moral

Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh
imoral223489@bscse.uiu.ac.bd

Dewan Md. Farid

Professor, Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh
dewanfarid@cse.uiu.ac.bd

Abstract—Naïve Bayes (NB) can filter training instances before a decision tree is fitted, and a decision tree can select and weight attributes for NB. We join these two steps into Algorithm 3 and run them in both orders, with either NB or C4.5 as the final classifier. All methods are tested on ten benchmark datasets with ten repetitions of stratified 10-fold cross-validation. Each cleaning step is fitted on the training fold only, and every test row is scored. The two combined tree methods reach 81.21% and 80.89% mean accuracy, against 86.33% for C4.5 without cleaning. The combined NB methods reach 75.63% and 76.35%, against 75.93% for plain NB and 77.35% for attribute weighting alone. Paired tests over the datasets show no clear accuracy difference between the two orders. When the filter is fitted on the full dataset before cross-validation, the mean tree accuracy of Algorithm 1 rises by 13.50 percentage points, but the filter also removes cases that would later be tested. The removal records show that the filter can drop a much larger share of one class than of another. On these datasets we found no evidence that combining the steps improves on the base classifiers, and the scores cannot be read correctly without knowing where cleaning took place and which cases it removed.

Index Terms—Naïve Bayes, decision tree, instance filtering, attribute weighting, hybrid classifier, cross-validation

I. INTRODUCTION

Farid et al. [1] introduced two hybrids that connect Naïve Bayes (NB) and decision trees. Algorithm 1 trains NB on the training set, removes the instances it misclassifies, and grows a tree on the remaining rows. Algorithm 2 grows a tree first, keeps the attributes it tests, and gives more weight to attributes tested closer to the root when fitting NB. In each method, one classifier prepares the data for the other.

Running both steps is the obvious extension, but combining them changes the data each learner sees. If instances are removed first, the tree picks attributes from a smaller sample. If attributes are picked first, a weighted NB judge decides which instances stay. We call the combined procedure Algorithm 3 and test both orders. We report a final tree and a final NB separately, so that a gain from switching classifiers is not mistaken for a gain from cleaning.

Where the cleaning happens also changes what a score means. A filter fitted before cross-validation can see the labels

of rows that will later be test rows, and it can remove those rows before they are ever scored. A filter fitted inside each training fold leaves every test row in place. The first approach therefore changes both the information the cleaner uses and the set of rows on which accuracy is measured.

We compare Algorithm 3 with each single step and with the uncleaned baselines, using the same fold-specific protocol for all methods. We report accuracy, macro-F1, and the share of each class removed by the filter, and we compare cleaning inside the training fold with cleaning before the split. On this benchmark the combined methods show no clear advantage. The contribution of this paper is a controlled comparison, together with an account of how the evaluation protocol and the removed rows affect the reported scores.

II. RELATED WORK

Hall [2] used decision trees to derive attribute weights for NB as a way to reduce the effect of its conditional-independence assumption. Farid et al. [1] proposed the two steps studied here and evaluated each one on its own. Algorithm 3 chains those steps and keeps the same tree learner and NB likelihood model.

Instance filtering needs more care. Brodley and Friedl [3] used several classifiers to identify mislabeled training data. When a single NB judge disagrees with a label, that alone does not show the label is wrong. In this paper, a “filtered” row is one the judge rejected. We make no claim that the removed rows contain real label errors.

Cawley and Talbot [4] showed that evaluation choices can introduce selection bias large enough to affect comparisons between learning algorithms. We ask the same kind of question about where supervised cleaning is placed, for one family of hybrids. Following Demšar [5], we make paired comparisons at the dataset level. The repeated folds estimate each dataset’s score and are not treated as independent samples.

III. ALGORITHMS

A. Algorithm 1: instance filtering

Given training data D , the NB judge is fitted on D and predicts the labels of those same rows. A row is kept only if the prediction matches its given label. Algorithm 1 then fits C4.5 on the kept rows. The reference arm that ends in NB uses the same filter but fits NB again as the final classifier. If deletion would leave fewer than two classes, our implementation keeps the original training rows.

This is a disagreement rule on the training set, so its predictions are not out-of-fold estimates of label quality. A judge can reject a correctly labeled case when the classes overlap or when its own model assumptions do not fit the data.

B. Algorithm 2: attribute selection and weighting

A C4.5 tree is fitted on the training data. Let d_j be the smallest depth at which it tests attribute j , with the root at depth one. The weight is

$$w_j = \begin{cases} 1/\sqrt{d_j}, & j \text{ is tested,} \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

Attributes the tree does not test are dropped. The weighted NB class score is

$$s_c(x) = \log P(c) + \sum_{j \in S} w_j \log P(x_j | c), \quad (2)$$

where S is the set of kept attributes. NB predicts the class with the largest score. The reference arm that ends in a tree keeps only the selected columns, and its final C4.5 does not use the weights.

C. Algorithm 3: the combined hybrid

The instances-first path, NA, first runs the plain NB filter of Algorithm 1. It then fits a tree on the kept rows to obtain the selected attributes and their weights. The attributes-first path, AN, takes the attributes and weights from the full training fold and then uses weighted NB to filter the rows. In AN, the judge and the final NB use the same weights, and the weights are not estimated again after deletion. Each path ends in either weighted NB or C4.5.

Algorithm 3 uses the same filtering safeguard as Algorithm 1. If the selection tree tests no attribute, the combined path keeps all current columns with unit weights. This never happened in the stored runs. At prediction time, the test fold keeps only the columns in S but all of its rows, so every test case is classified.

IV. EXPERIMENTAL METHOD

A. Datasets and implementation

We use the ten datasets named in the 2014 study. Table I lists the sizes of the copies we used. Image segmentation joins the two UCI files, and NSL-KDD uses its 20% training file [6]. Our Glass and NSL-KDD files each have one class fewer than the original study lists. The data come from the same

Algorithm 3 Combined cleaning and classification

Require: Training data D , order $o \in \{\text{NA}, \text{AN}\}$, final learner $L \in \{\text{NB}, \text{DT}\}$

- 1: **if** $o = \text{NA}$ **then**
- 2: $D \leftarrow \text{FILTER}(D, \text{plain NB})$
- 3: $(S, W) \leftarrow \text{SELECT}(D, \text{C4.5})$
- 4: **else**
- 5: $(S, W) \leftarrow \text{SELECT}(D, \text{C4.5})$
- 6: $D \leftarrow D[:, S]$
- 7: $D \leftarrow \text{FILTER}(D, \text{NB with } W)$
- 8: **end if**
- 9: **if** $o = \text{NA}$ **then**
- 10: $D \leftarrow D[:, S]$
- 11: **end if**
- 12: Fit M on D using L (use W if $L = \text{NB}$)
- 13: **return** M, S

TABLE I
DATASET SIZES IN THE FILES USED FOR EVALUATION

Dataset	Rows	Attributes	Classes
Breast cancer	286	9	2
Contact lenses	24	4	3
Diabetes	768	8	2
Glass	214	9	6
Iris	150	4	3
Soybean	683	35	19
Vote	435	16	2
Image segmentation	2310	19	7
Tic-tac-toe	958	9	2
NSL-KDD	25192	41	22

benchmark family, but this is not an exact reproduction of the original data or splits.

C4.5 is Weka J48 version 3.8.6 with pruning enabled and the default settings $-C 0.25 -M 2$ [7]. Our NB uses add-one counts for nominal attributes and Gaussian likelihoods for numeric attributes. Nominal attributes stay as single columns, so a tree selects an original attribute rather than separate one-hot indicators. The category list of each nominal attribute comes from the dataset file and is fixed across folds. A missing nominal value is treated as its own category. Counts, numeric distributions, and class priors are estimated from the training rows alone.

B. Protocols and comparison arms

Protocol B is our main evaluation. We run stratified 10-fold cross-validation with seeds 0–9, which gives 100 scored folds per dataset and method. All methods use the same splits. The judge, the selection tree, and the final classifier are all fitted on the training rows of each split. Only the choice of columns is carried over to the test fold, and test rows are never filtered. This is repeated cross-validation with preprocessing inside each fold. It is not a nested search over hyperparameters.

Protocol A fits the cleaning steps once on the whole dataset and then runs cross-validation for the final classifier on the cleaned data. The cleaner therefore sees the labels of future test

rows. For methods that filter instances, it also removes future test rows, so the folds are built from a different sample. We report these score changes as descriptive results only. Protocol A folds are not paired with Protocol B folds, and the difference between the two cannot separate label leakage from the change in which rows are evaluated.

The comparison includes plain NB and C4.5, the original Algorithm 1–DT and Algorithm 2–NB methods, and the four versions of Algorithm 3. Two reference arms help show where changes come from. Algorithm 1–NB changes only the final learner after filtering, and Algorithm 2–DT applies attribute selection before another tree. In the tables, N stands for filtering, A for attribute selection and weighting, and NA and AN for the two combined orders.

C. Measures and paired analysis

For each method and dataset, accuracy and macro-F1 are averaged over the 100 scored folds, with equal weight for each fold. Within a fold, macro-F1 averages the F1 scores of the classes that appear in the true labels or the predictions, and a class with no correct prediction counts as zero. The summary figures then give each of the ten datasets equal weight, so they are not pooled scores over all instances.

We compare eight pairs with two-sided Wilcoxon signed-rank tests on the ten dataset-level mean accuracies. The pairs are the two original hybrids against their baselines, each of the four combined methods against its baseline, and the two order comparisons. We compute exact p values by enumerating all sign permutations, which suits the small sample and handles zero differences. Holm adjustment controls the family of eight tests at 0.05. We chose these tests after the experiments had been run and applied them to the saved results, so they should be read as exploratory. Mean differences are given in percentage points (pp). Differences between the combined methods and the single-step methods are reported as means only, without tests. A nonsignificant result does not show that two methods are equivalent.

For each filtering step we also record the share of each class that is removed, using that class’s number of training rows as the denominator. This shows how deletion changes the class balance without assuming that the removed labels were wrong.

V. RESULTS AND DISCUSSION

A. Combined methods have lower mean tree accuracy

Table II gives the averages over the benchmark, and Table III gives the results for each dataset. Uncleaned C4.5 reaches 86.33% accuracy and 81.92% macro-F1. The two combined tree methods reach 81.21% and 80.89% accuracy, which is 5.12 and 5.44 pp lower. Their macro-F1 also drops, to 75.76% and 73.93%. Compared with the 81.03% of Algorithm 1, adding attribute selection after filtering gains only 0.18 pp.

For NB, attribute weighting alone has the highest mean accuracy, 77.35% against 75.93% for plain NB. The combined methods reach 75.63% and 76.35%, or -0.30 and $+0.42$ pp relative to plain NB, and neither one matches Algorithm 2.

TABLE II
PROTOCOL B RESULTS (%), AVERAGED EQUALLY ACROSS TEN DATASETS

Method	Accuracy	Macro-F1
NB	75.93	69.80
C4.5	86.33	81.92
Algorithm 1	81.03	75.70
Algorithm 2	77.35	71.73
Instances first, NB	75.63	69.60
Instances first, DT	81.21	75.76
Attributes first, NB	76.35	69.91
Attributes first, DT	80.89	73.93

The reference arms point to filtering as the main source of the loss. Filtering followed by NB reaches 75.42%, while selecting attributes before C4.5 leaves its mean almost unchanged at 86.19%.

The averages hide large differences between datasets. On Glass, instances-first NB falls from 45.95% for plain NB to 30.93%, and instances-first C4.5 falls from 66.58% to 48.97%. On Contact lenses, the same NB combination rises from 73.00% to 83.00%, but that dataset has only 24 rows. A cleaner should be checked dataset by dataset before it is called helpful.

Table IV reports the paired accuracy tests. AN–DT is below the C4.5 baseline with raw $p = 0.014$, but its Holm-adjusted $p = 0.109$ is above the threshold. None of the eight comparisons is significant after adjustment. The mean losses are still worth noting, but this benchmark does not show a general advantage or disadvantage across classification problems.

Switching from NA to AN raises mean NB accuracy by 0.71 pp and lowers mean tree accuracy by 0.32 pp, with raw p values of 0.275 and 0.922. We see no clear order effect under these settings. This result is limited to a single pass through the two steps, with fixed learner settings and with the AN judge using the selection weights.

B. Cleaning before the split changes the score

Table V compares the two protocols. Algorithm 1 reaches 94.52% when the filter is fitted on the full data, against 81.03% when the judge is fitted inside each training fold. That is an apparent gain of 13.50 pp. Algorithm 2 changes much less, from 77.35% to 78.07%. The combined tree methods also jump under Protocol A, to 94.72% and 96.32%.

The two protocols measure different things. Protocol B asks the final classifier to predict every held-out case. Protocol A first lets the NB judge decide which cases reach the test folds. The large increase fits with the judge removing cases it finds hard, even if the final classifier does no better on the full data. Attribute-only cleaning keeps all rows, but selecting attributes on the full data still uses information from future test folds. Its small score change here does not mean that attributes can safely be selected before cross-validation.

C. Deletion changes class balance

The breast-cancer records show how uneven the removal can be. Across Protocol B folds, the plain NB judge removes on

TABLE III

PROTOCOL B ACCURACY BY DATASET (%). BASE IS THE UNCLEANNED CLASSIFIER; N IS INSTANCE FILTERING; A IS ATTRIBUTE SELECTION AND WEIGHTING; NA AND AN ARE THE TWO ALGORITHM 3 ORDERS. N-NB AND A-DT ARE REFERENCE ARMS.

Dataset	Final NB					Final C4.5				
	Base	N	A	NA	AN	Base	N	A	NA	AN
Breast cancer	72.12	72.68	72.43	71.28	72.53	74.84	71.77	73.91	71.77	72.08
Contact lenses	73.00	73.00	80.67	83.00	79.33	84.67	84.67	84.67	84.67	80.67
Diabetes	75.52	74.41	76.73	75.58	75.67	74.12	75.12	74.18	75.23	75.57
Glass	45.95	37.00	47.84	30.93	37.28	66.58	48.83	66.58	48.97	49.15
Segmentation	79.76	74.92	82.30	74.53	77.32	96.81	89.24	96.83	89.26	90.88
Iris	95.53	95.33	95.40	95.33	95.87	94.27	94.93	94.27	94.93	93.80
NSL-KDD	67.42	75.79	62.80	74.53	75.23	99.47	87.43	99.47	89.68	87.07
Soybean	90.09	89.00	89.40	85.67	86.66	92.71	88.71	92.26	87.89	90.98
Tic-tac-toe	69.83	71.89	70.68	70.03	68.07	84.74	74.43	84.74	74.55	73.17
Vote	90.06	90.13	95.24	95.45	95.49	95.08	95.13	95.03	95.13	95.54

TABLE IV

DATASET-LEVEL PAIRED ACCURACY TESTS. DIFFERENCES ARE FIRST METHOD MINUS SECOND; HOLM ADJUSTMENT COVERS ALL EIGHT COMPARISONS.

Comparison	Δ (pp)	p	Holm p
N-DT vs. DT	-5.30	0.055	0.383
A-NB vs. NB	+1.42	0.131	0.654
NA-DT vs. DT	-5.12	0.055	0.383
AN-DT vs. DT	-5.44	0.014	0.109
NA-NB vs. NB	-0.30	1.000	1.000
AN-NB vs. NB	+0.42	0.846	1.000
AN-DT vs. NA-DT	-0.32	0.922	1.000
AN-NB vs. NA-NB	+0.71	0.275	1.000

TABLE V

MEAN ACCURACY (%) UNDER THE TWO PROTOCOLS. FILTERING BEFORE CV CHANGES WHICH ROWS ARE EVALUATED, SO THESE SCORES SHOULD BE READ AS A DIAGNOSTIC COMPARISON.

Method	Fold-specific	Before CV	Difference
Algorithm 1	81.03	94.52	+13.50
Algorithm 2	77.35	78.07	+0.72
Alg. 1, NB final	75.42	91.93	+16.51
Alg. 2, DT final	86.19	86.52	+0.33
Instances first, NB	75.63	90.11	+14.48
Instances first, DT	81.21	94.72	+13.51
Attributes first, NB	76.35	91.08	+14.74
Attributes first, DT	80.89	96.32	+15.43

average 50.63% of the recurrence-event rows but only 14.51% of the no-recurrence-event rows. After attribute selection, the weighted judge removes 58.97% and 11.72%. These are mean removal rates within each class, not shares of the whole dataset.

In both paths the minority class loses a much larger share of its training rows. The records cannot tell us whether those labels were wrong or whether NB simply failed on hard but valid cases. They do show that a cleaner should be judged by the rows it removes as well as by its scores.

VI. LIMITATIONS

The benchmark uses the dataset family of the original study, with the file differences noted above. It says nothing about

performance on modern datasets or on data with a known level of label noise. In particular, our NSL-KDD C4.5 baseline is 99.47%, while the 2014 study reports 71.11%. We have not been able to explain this gap. The comparison here uses shared data and splits, but it should not be described as an exact replication of the earlier scores.

Rare classes also limit the evaluation. Contact lenses has classes with four and five rows, and NSL-KDD has several classes with fewer than ten examples, four of which have a single row. As a result, some stratified folds miss a class in training or testing. Repeating the splits does not create new examples, and the per-fold macro-F1 can average over different class sets. These results do not support reliable claims about rare classes.

In Protocol B, model fitting and supervised cleaning use only training rows, but the category lists of nominal attributes come from the complete files. The estimates therefore assume these categories are known in advance, and they do not test unseen nominal values. We did not use calibration or out-of-fold judging to check the rejection decisions of the filter. With ten datasets, the tests can miss small effects. Finally, the Protocol A results cannot tell us which evaluation procedure the 2014 study used.

VII. CONCLUSION

Algorithm 3 combines NB instance filtering and tree-based attribute weighting in either order. On ten datasets with fold-specific evaluation, both combined tree methods have lower mean accuracy and macro-F1 than uncleaned C4.5, and both combined NB methods fall short of attribute weighting alone. The paired tests show no clear order effect. The before-CV scores and the removal rates per class lead to a practical recommendation. Supervised cleaning should be fitted inside the training fold so that every held-out row is still scored, and studies that use such a filter should report how many training cases each class loses.

REFERENCES

- [1] D. M. Farid, L. Zhang, C. M. Rahman, M. A. Hossain, and R. Strachan, "Hybrid decision tree and naïve Bayes classifiers for multi-class classification tasks," *Expert Systems with Applications*, vol. 41, no. 4, pp. 1937–1946, 2014.

- [2] M. Hall, "A decision tree-based attribute weighting filter for naive Bayes," *Knowledge-Based Systems*, vol. 20, no. 2, pp. 120–126, 2007.
- [3] C. E. Brodley and M. A. Friedl, "Identifying mislabeled training data," *Journal of Artificial Intelligence Research*, vol. 11, pp. 131–167, 1999.
- [4] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010.
- [5] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, no. 1, pp. 1–30, 2006.
- [6] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. IEEE, 2009, pp. 1–6.
- [7] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Cambridge, MA: Morgan Kaufmann, 2016.